Invited: Hardware/Software Co-Synthesis and Co-Optimization for Autonomous Systems

Wanli Chang*†, Shuai Zhao†, Simon Burton‡, Haitong Wang†, Ting Chen§, Nan Chen†, Neil Audsley†¶
*College of Computer Science and Electronic Engineering, Hunan University, China
†Department of Computer Science, University of York, UK
‡Research Division for Safety, Fraunhofer Institute for Cognitive Systems IKS, Germany
§School of Computer Science and Engineering, University of Electronic Science and Technology of China, China
¶School of Mathematics, Computer Science and Engineering, City, University of London, UK

2021 58th ACM/IEEE Design Automation Conference (DAC) | 978-1-6654-3274-0/21/$31.00 ©2021 IEEE | DOI: 10.1109/DAC18074.2021.9586147

***Abstract*—With ever more complicated functionalities being integrated in modern autonomous systems, traditional design methods may not remain sufficient to deliver trusted and high-performance systems with stringent temporal, safety and cost efficiency requirements. In this paper, we discuss the limitations of the traditional design methods with the above requirements enforced, in which hardware and software design are often considered separately. To tackle these limitations, this paper presents a novel design solution that synthesizes both software-level and hardware-level design. First, we highlight and analyze the interconnections between software-level methods (e.g. priority assignment and task allocation) and hardware design (e.g. cache and memory management), in terms of the resulting system performance, e.g. latency. Second, by applying the identified interconnections, we propose an optimization framework to produce high-quality synthesized solutions of both software and hardware design based on a set of candidate design methods. In addition, we describe potential research directions derived from the work and major challenges that can be investigated jointly by engineers and researchers from embedded systems, system safety and programming languages communities.**

I. Introduction

With the trend towards the emerging autonomous systems, complicated functionalities are integrated into the same multi-core platform to meet the increasing demand of applications. The design of a complete autonomous system requires both hardware-level (e.g. cache and memory management) and software-level (e.g. priority assignment and task allocation) design methods, with stringent resource limitations and high-standard performance requirements (e.g. dynamic resource sharing, latency, timing predictability, and cost efficiency).

Traditional design methods are no longer sufficient due to the limitation that hardware and software (H/S) designs are often considered separately. For example, Wang et al. [1] propose the Meshed Bluetree architecture to support multi-core systems with increased bandwidth, which allows simultaneous memory accesses to be processed by paralleled memory modules concurrently with time-predicable behavior [1]. However, the effectiveness of such a design depends on efficient strategies to map tasks to memory modules within the system, which is often decided by software-level task allocation methods.

Ting Chen is the corresponding author (brokendragon@uestc.edu.cn).

In addition, software design seldom interacts with the underlying hardware. For example, Zhao et al. [2] propose the Slack-based Priority Ordering to improve system schedulability with the consideration of resource contention. However, the algorithm is developed at the software-level only and neglects underlying contention due to hardware-level components, e.g. memory and bus, which would undermine the effectiveness of the algorithm. Besides, the execution time is not analyzed with the integration of the computing architecture, which can be affected by hardware-level methods and in turn, imposes potential impacts on the execution time of tasks. As a consequence of not considering the H/S design methods synthetically, once they are applied together, the overall performance can be significantly beneath expectations due to potential collisions. In this case, the above traditional design methods cannot deliver trusted and high-performance systems with the ever increasing requirements.

Therefore, the impediment of the compartmentalized design procedure has facilitated the development of the H/S co-design methods. Hardware vendors have started to embrace the development of software based on their own hardware architecture to remain competitive. For example, Intel has developed machine learning software that are optimized to fit the underlying hardware and to offer maximized performance on Intel architecture [3]. In addition, Apple has developed its own silicon chips in the latest PC products for better software technology deployment [4]. However, an effective co-design solution need to be formalized to confront the escalation of software and hardware evolution.

This paper proposes a novel design method that co-synthesizes and co-optimizes H/S design solutions to achieve maximized system performance. First, we introduce the notion of the *H/S interconnections*, which is represented as a multidimensional correlation model between hardware and software design solutions. The *H/S inter-connections* can analyze the resulting system performance when certain H/S design solutions are applied to a system. Second, depending on the *H/S interconnections*, we present a novel H/S co-synthesis and co-optimization framework. Based on the system speci-

fication, the framework first performs the co-synthesis process that initializes a set of compatible design combinations from hardware and software candidate methods. Subsequently, an optimization procedure is performed based on the performance analysis through modifications of the selection. The finalized output design combination of hardware and software aims to provide optimal H/S solutions that satisfy the specification.

Furthermore, this paper describes a potential direction with insights for eliminating the design barriers exist in the conventional design approach and maximizing the system performance. Major challenges derived from the work are identified and discussed, which can be investigated jointly by engineers and researchers from embedded systems, system safety and programming languages communities.

This paper is organized as follows. Section II describes the major components in hardware and software design, and addresses the necessities and corresponding research challenges of establishing the H/S interconnections. Section III demonstrates a complete procedure of the proposed H/S co-synthesis and co-optimization framework based on the interconnections. Section IV draws the conclusion.

II. H/S Design Methods and Interconnections

This section describes the major H/S design components in a system and the interconnections exist between H/S design solutions. Major research challenges for analyzing the interconnections are identified with potential solutions discussed.

A. Major H/S design components

The conventional design procedure of autonomous systems often focuses on software and hardware independently. At the software level, innumerable design methods can influence the system performance such as the makespan (i.e., the execution time between the start of the first task and end of the last task) and predictability. As described in [5], the software design methods can be generally categorized as: *scheduling policies* (e.g. Fully Partitioned Fixed Priority Scheduling and Earliest Deadline First Scheduling schemes), *task allocation schemes* (e.g. Worst-fit and Best-fit algorithms), *priority assignments* (e.g. Deadline Monotonic and Rate Monotonic priority ordering) as well as *resource sharing protocols* (e.g. Multiprocessor Stack Resource Protocol [6] and Multiprocessor resource sharing Protocol [7]). Each candidate method in the above categories imposes certain behaviors of the system, and hence, results in varied system performance.

At the hardware-level, both of the architectural structure (e.g. shared or distributed memory system) and management protocols (e.g. cache replacement and bus arbitration policies) of these components are significant to the expense and performance of the system. For example, the addition of extra cores or RAMs will increase the cost of the hardware directly. In addition, different cache replacement policies (e.g. round-robin and FIFO) can be applied according to the divergent system contexts to minimize cache misses and hence improve system performance.

More importantly, the H/S design methods are intimately related to each other and have a joint effect on the execution of the system [8]. For instance, simply increasing the number of cores at hardware-level would increase system parallelism. However, without effective task allocation and resource control algorithms, this can also intensify the contention for accessing shared resources, which would cause additional blocking time and undermine the effectiveness of the design choice. Similarly, if the task allocation and priority assignment algorithms are selected without considering the underlying hardware, the timing predictability of the system is likely to be jeopardized due to contentions in the underlying hardware components, e.g. memory, bus and cache. Therefore, the H/S co-synthesis design is becoming ever more important to achieve maximized system performance, especially for modern autonomous systems which often have limited resources with stringent timing, expenses and energy requirements.

B. Interconnections of H/S design methods

To enable H/S co-synthesis design, the collective effects of H/S design methods on the system should be analyzed and understood. To achieve this, we model the co-effects of H/S design methods as *H/S interconnections*. As shown in Figure 1, the system contains a set of H/S design components, each of which has several candidate design choices, e.g. different scheduling methods and the number of cores.

At the software level, the execution scenario of tasks in the system can be effectively obtained with the design methods decided, and hence, are considered in a holistic manner. For each hardware component, a H/S interconnection is created to produce the effects of its design choice to the system performance along with the software-level methods. For example, the interconnection between *# of cores* and the software should produce the end-to-end latency when a given number of cores and certain software-level design methods. Furthermore, the H/S interconnections can be extended to a multi-dimensional model to provide measurements of different metrics, e.g. latency, energy consumption and cost of hardware, which are common requirements for modern autonomous systems.

The H/S interconnections provide the basis for evaluating different design choice combinations and enabling the best match of H/S design choices. However, it is challenging to construct the interconnections so that they are flexible enough to support the analysis of different candidate design methods. This leads to the first research challenge of this work:

C.1 *How to establish flexible H/S interconnections to support the analysis of multiple H/S candidate methods?*

A potential solution is to apply the static analysis. For example, the response time analysis (RTA) [9] can be applied to construct one of the dimensions of the interconnection between software and # of cores (*S-P* in Figure 1), which computes the worst-case response time of each task. Equation (1) is a fundamental version of RTA where C_i denotes the computation of a task τ_i, B_i indicates the blocking time incurred by τ_i, and $\sum_{j \in \mathbf{hp}(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$ represents the interference of τ_i from high priority tasks on the same core (i.e.

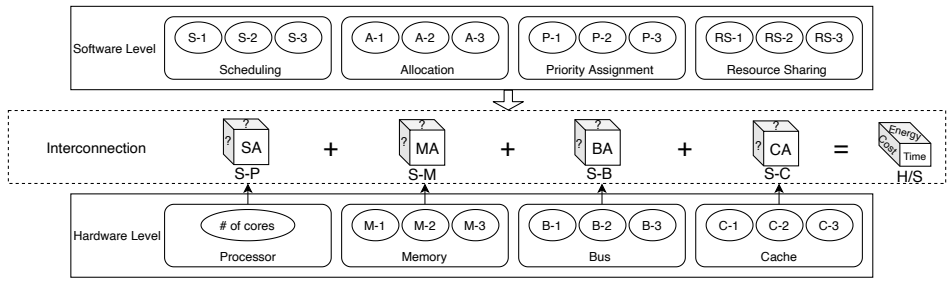

Fig. 1: The H/S interconnections of hardware and software components in a system

lhp(i)). This equation can be computed giving a task allocation scheme, a priority assignment and a number of cores. An extended version of RTA is given in [10] to include the analysis of different resource control protocols.

$$R_i = C_i + B_i + \sum_{j \in \mathbf{lhp}(i)} \left\lceil \frac{R_i}{T_j} \right\rceil \times C_j \tag{1}$$

In addition, analysis for the power consumption [11] and hardware cost can be applied to construct other dimensions of the H/S interconnection (e.g. the 3D *S-P* interconnection), which provide comprehensive measurements of metrics that are crucial for autonomous systems, e.g. latency, energy consumption and hardware cost. Following the same approach, interconnections between software and other hardware components can also be constructed using static analysis, e.g. software and memory (bus) with analysis supporting different arbitration policies (see Figure 1).

However, as modern autonomous systems are becoming ever more complex, it is increasingly difficult to statically analyze a system, for example, the cache miss in a shared cache of a large system. Under this case, data-driven learning methods become attractive as they focus on high-level system behaviors instead of examining each basic block in the system [12]. With learning-based methods, a model can be trained to produce performance predictions given different design choices based on the key differences of candidate solutions [13]. Due to space limitation, we do not present details of the learning-based approach and refer interested readers to [12] and [13].

Once the interconnections are constructed for the software and each individual hardware component, the co-effects between the interconnections should be analyzed to obtain precise system performance results. The reason for such an analysis is that a design choice made for a hardware component can affect task behaviors, which in turn, affects the performance of other hardware components. For instance, a well-developed cache replacement policy can decrease the cache misses in the system and thereby result in less memory access demand and bus contention [14]. In contrast, an increased number of cores would increase system parallelism but can also intensify the memory and bus contention. Establishing such correlations is a key to obtain precise system performance results and to enable maximized performance. However, unlike the individual H/S interconnections, analyzing the correlations between interconnections would involve multiple software and hardware components. In addition, the impacts between these components are often bidirectional and their correlations are relatively unintuitive. The above significantly complicates the analysis. This raises the second research challenge:

C.2 *How to establish the correlations between individual H/S interconnections?*

For correlations that can be effectively expressed by static analysis (e.g. the one between bus contention and # of cores), the established analysis can be extended to include both hardware components, e.g. the CAN bus analysis in [15]. However, for convoluted scenarios (e.g. cache and memory), data-driven methods can be applied to establish a learned model that reflects the underlying correlations [13].

Finally, regardless of the approach being applied for constructing the interconnections, scalability is a key concern towards applicability. This is because analyzing the interconnection can be a resource-intensive and error-prone process if the interconnections cannot be easily extended to include new design methods or S/H components (e.g. I/O and networking), which is inappropriate in real-world application scenarios. This raises the third research challenge:

C.3 *How to enable scalability of the H/S interconnections?*

A systematic and automated procedure should be constructed to fasten the construction of new interconnections and the extension of the existing interconnections when an extra design element is introduced. In particular, there is a research direction which applys the evolutionary algorithms to semi-automate the process of deriving response time analysis [16]. This research can be further advanced to lessen the complexity of static analysis for establishing the interconnections. Furthermore, regarding the data-driven learning modeling, a full automated process of data gathering, data analysis and model learning would significantly reduce the time spends on re-constructing the interconnections.

III. Hardware/Software Co-Synthesis and Co-Optimization Framework

Based on the H/S interconnections, we propose a multi-objective H/S co-synthesis and co-optimization framework to deliver complete H/S co-design solutions for a given system. Figure 2 illustrates the working mechanism of the proposed framework. The framework can be constructed with a wide range of optimization algorithms, e.g. simulated annealing and

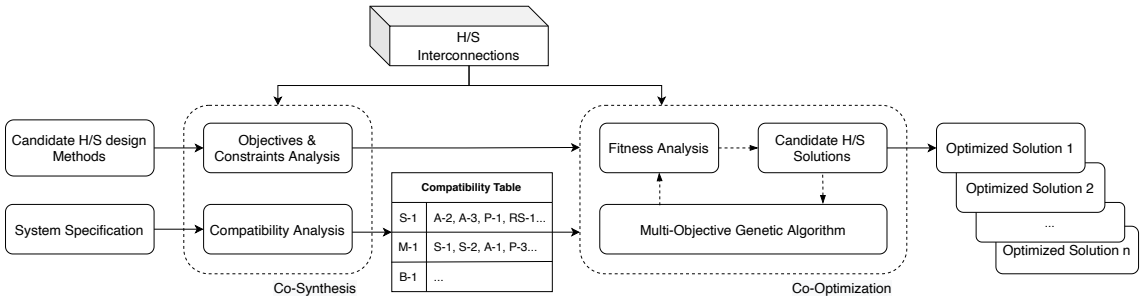

Fig. 2: Co-Synthesis and Co-Optimization Framework to Deliver System Solutions.

dynamic programming. In this paper we use the Genetic Algorithm (GA) as an example to illustrate its working mechanism.

The proposed framework includes two major processes. First, the co-synthesis process 1) analyzes the system specification to obtain a set of optimization objectives and constraints (e.g. hard deadlines, security, safety, energy consumption and hardware cost), and 2) iterates each candidate design method to find its compatible solutions in other design components (i.e. the compatibility table). During this process, the H/S interconnections are required to check whether a certain optimization objective derived from the specification is supported by the current version of interconnections. In addition, the interconnections can be used to investigate the compatibility of two design solutions, where a large penalty value (e.g. -1000) can be returned for H/S design solutions that are not compatible. This step is beneficial for the optimization process as some infeasible H/S synthesized solutions can be eliminated before the optimization process.

Based on the optimization objectives, constraints and the compatibility table, the co-optimization process generates a set of candidate H/S solutions and preforms GA-based optimization to improve the solutions iteratively (i.e. by generations). The H/S interconnections in this process serves as the fitness function to produce measurements for the obtained objectives and constraints. For example, the RTA and CAN analysis described in Section II can be integrated together to compute *latency* when design methods of software, processors and bus are synthesized. In addition, the penalty value will be returned if a synthesized solution cannot satisfy the system constraints. The termination criteria of optimization framework can be configured by the maximum number of generations. Once the optimization is finished, a Pareto Front approximation can be obtained which contains a set of non-dominated H/S co-designs that are favorable for one or more objectives.

IV. Conclusion

This paper describes a novel design method that co-synthesizes and co-optimizes the H/S design solutions, to eliminate barriers exist in the traditional system design approach and to provide maximized system performance. First, the interconnections between H/S design methods are identified and analyzed, which captures impacts on system performance from interactions of different H/S design methods. Based on the interconnections, an optimization framework is presented to provide high-quality synthesized H/S design solutions among the a set of candidate design methods. Major research challenges are identified with potential research directions explained. In future work, we will establish the described H/S interconnections, implement the proposed optimization framework and evaluate its efficacy against the traditional system design approach.

References

[1] H. Wang, N. C. Audsley, X. S. Hu, and W. Chang, "Meshed bluetree: Time-predictable multimemory interconnect for multicore architectures," *IEEE TCAD*, 2020.

[2] S. Zhao, W. Chang, R. Wei, W. Liu, N. Guan, A. Burns, and A. Wellings, "Priority assignment on partitioned multiprocessor systems with shared resources," *IEEE TC*, 2020.

[3] A. V. Cueva, "Intel's optimized tools and frameworks for machine learning and deep learning," 2016. [Online]. Available: https://software.intel.com/content/www/us/en/develop/home.html

[4] R. Iyengar, "Apple details new macbook air, macbook pro and mac mini – all powered by in-house silicon chips," 2020. [Online]. Available: https://edition.cnn.com/2020/11/10/tech/apple-silicon-chips-mac/index.html

[5] R. I. Davis and A. Burns, "A survey of hard real-time scheduling for multiprocessor systems," *ACM CS*, 2011.

[6] P. Gai, G. Lipari, and M. Di Natale, "Minimizing memory utilization of real-time task sets in single and multi-processor systems-on-a-chip," in *RTSS*, 2001.

[7] A. Burns and A. J. Wellings, "A schedulability compatible multiprocessor resource sharing protocol – MrsP," in *ECRTS*, 2013.

[8] R. Ernst, D. Ziegenbein, K. Richter, L. Thiele, and J. Teich, "Hardware/software codesign of embedded systems the spi workbench," in *in VLSI'99*, 1999.

[9] N. Audsley, A. Burns, M. Richardson, K. Tindell, and A. J. Wellings, "Applying new scheduling theory to static priority pre-emptive scheduling," *IET SE*, 1993.

[10] S. Zhao, "A FIFO spin-based resource control framework for symmetric multiprocessing," Ph.D. dissertation, 2018.

[11] S. Pagani and J.-J. Chen, "Energy efficiency analysis for the single frequency approximation (SFA) scheme," *ACM TECS*, 2014.

[12] D. Griffin, B. Lesage, I. Bate, F. Soboczenski, and R. I. Davis, "Forecast-based interference: Modelling multicore interference from observable factors," in *RTNS*, 2017.

[13] X. Dai, S. Zhao, I. Bate, A. Burns, X. Guo, and W. Chang, "Brief industry paper: Digital twin for dependable multi-core real-time systems - requirements and open challenges," in *RTAS*, 2021.

[14] S. A. Rashid, G. Nelissen, and E. Tovar, "Cache persistence-aware memory bus contention analysis for multicore systems," in *DATE*, 2020.

[15] R. I. Davis, A. Burns, R. J. Bril, and J. J. Lukkien, "Controller area network (CAN) schedulability analysis: Refuted, revisited and revised," *Springer RTS*, 2007.

[16] P. Dziurzanski, R. I. Davis, and L. S. Indrusiak, "Synthesizing real-time schedulability tests using evolutionary algorithms: A proof of concept," in *RTSS*, 2019.